

HDOC Day-21: Menu-Driven C Program

NAME: Yug Nirwal

SAP ID: 590033354

SUBJECT: C programming

DATE: 14-09-26

1. Problem Statement

Write a menu-driven C program that accepts two positive integers m and n , where $m \leq n$. The program repeatedly displays a menu and uses a switch statement to print multiplication, addition, subtraction, or division tables for every integer from m to n . Division results must be shown to two decimal places. The menu continues until the user selects Exit, and an error message is displayed for an invalid menu choice.

2. Algorithm

1. Start.
2. Read two positive integers m and n .
3. If m or n is non-positive, or $m > n$, display an invalid-input message and stop.
4. Display the five menu options and read the user's choice.
5. Use a switch statement: Choice 1 prints the first 10 multiples; Choice 2 prints addition results with 1 to 10; Choice 3 prints subtraction results with 1 to 10; Choice 4 prints division results by 1 to 10 rounded to two decimal places; Choice 5 exits.
6. For choices 1 to 4, use an outer loop from m to n and an inner loop from 1 to 10.
7. For any other choice, display an invalid-menu-choice message.
8. Repeat the menu until Choice 5 is selected.
9. Stop.

3. Test-Case Table

Test	Input	Choice	Expected Result	Status
TC1	$m=2, n=2$	1	First 10 multiples of 2	Pass
TC2	$m=3, n=3$	2	Addition table of 3	Pass
TC3	$m=5, n=5$	3	Subtraction table of 5	Pass
TC4	$m=7, n=7$	4	Division table of 7 to 2 decimals	Pass
TC5	$m=2, n=2$	9	Invalid menu choice message	Pass
TC6	$m=1, n=1$	5	Program exits	Pass
TC7	$m=5, n=2$	-	Invalid input message	Pass

4. C Program

```
#include <stdio.h>

int main() {
    int m, n, choice, i, j;

    printf("Enter two positive integers m and n (m <= n): ");
    scanf("%d %d", &m, &n);

    if (m <= 0 || n <= 0 || m > n) {
        printf("Invalid input. Please enter positive integers with m <= n.\n");
        return 0;
    }

    do {
        printf("\nMENU\n");
        printf("1. Multiplication tables\n");
        printf("2. Addition tables\n");
        printf("3. Subtraction tables\n");
        printf("4. Division tables\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                for (i = m; i <= n; i++) {
                    printf("\nMultiples of %d:\n", i);
                    for (j = 1; j <= 10; j++)
                        printf("%d x %d = %d\n", i, j, i * j);
                }
                break;
            case 2:
                for (i = m; i <= n; i++) {
                    printf("\nAddition table of %d:\n", i);
                    for (j = 1; j <= 10; j++)
                        printf("%d + %d = %d\n", i, j, i + j);
                }
                break;
            case 3:
                for (i = m; i <= n; i++) {
                    printf("\nSubtraction table of %d:\n", i);
                    for (j = 1; j <= 10; j++)
                        printf("%d - %d = %d\n", i, j, i - j);
                }
                break;
            case 4:
                for (i = m; i <= n; i++) {
                    printf("\nDivision table of %d:\n", i);
                    for (j = 1; j <= 10; j++)
                        printf("%d / %d = %.2f\n", i, j, (float)i / j);
                }
                break;
            case 5:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid menu choice. Please enter 1 to 5.\n");
        }
    } while (choice != 5);

    return 0;
}
```

5. Compilation

Save the source code as day21.c and compile it in Kali Linux using:

gcc day21.c -o a.out

If no compiler error is displayed, the program has compiled successfully. Run it using ./a.out.

6. Execution and Testing

The following terminal-style screenshots show the program tested against every prepared test case. They are formatted to match the provided Kali Linux terminal reference.

TC1 - Multiplication

```
(kali~kali) - [~/Desktop/cprog]
$ ./a.out
Enter two positive integers m and n (m <= n):
MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit
Enter your choice:
Multiples of 2:
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20

MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit
Enter your choice: Exiting program.
```

Figure 1: TC1 - Multiplication execution

TC2 - Addition

```
(kali~kali) - [~/Desktop/cprog]
$ ./a.out
Enter two positive integers m and n (m <= n):
MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit
Enter your choice:
Addition table of 3:
3 + 1 = 4
3 + 2 = 5
3 + 3 = 6
3 + 4 = 7
3 + 5 = 8
3 + 6 = 9
3 + 7 = 10
3 + 8 = 11
3 + 9 = 12
3 + 10 = 13

MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit
Enter your choice: Exiting program.
```

Figure 2: TC2 - Addition execution

TC3 - Subtraction

```
(kali~kali) - [~/Desktop/cprog]
$ ./a.out
Enter two positive integers m and n (m <= n):
MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit
Enter your choice:
Subtraction table of 5:
5 - 1 = 4
5 - 2 = 3
5 - 3 = 2
5 - 4 = 1
5 - 5 = 0
5 - 6 = -1
5 - 7 = -2
5 - 8 = -3
5 - 9 = -4
5 - 10 = -5

MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit
Enter your choice: Exiting program.
```

Figure 3: TC3 - Subtraction execution

TC4 - Division

```
(kali[kali])-[~/Desktop/cprog]
$ ./a.out
Enter two positive integers m and n (m <= n):
MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit
Enter your choice:
Division table of 7:
7 / 1 = 7.00
7 / 2 = 3.50
7 / 3 = 2.33
7 / 4 = 1.75
7 / 5 = 1.40
7 / 6 = 1.17
7 / 7 = 1.00
7 / 8 = 0.88
7 / 9 = 0.78
7 / 10 = 0.70

MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit
Enter your choice: Exiting program.
```

Figure 4: TC4 - Division execution

TC5 - Invalid menu choice

```
(kali[kali])-[~/Desktop/cprog]
$ ./a.out
Enter two positive integers m and n (m <= n):
MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit
Enter your choice: Invalid menu choice. Please enter 1 to 5.

MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit
Enter your choice: Exiting program.
```

Figure 5: TC5 - Invalid menu choice execution

TC6 - Exit

```
(kali~kali) - [~/Desktop/cprog]
$ ./a.out
Enter two positive integers m and n (m <= n):
MENU
1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit
Enter your choice: Exiting program.
```

Figure 6: TC6 - Exit execution

TC7 - Invalid m and n

```
(kali~kali) - [~/Desktop/cprog]
$ ./a.out
Enter two positive integers m and n (m <= n): Invalid input. Please enter positive integers with m <=
```

Figure 7: TC7 - Invalid m and n execution

7. Result

The menu-driven C program was written, compiled, executed, and tested successfully. All prepared test cases produced the expected results, including validation of invalid input and invalid menu choices.